

Sistema RFID com ESP32 + MQTT + FastAPI

Objetivo do Projeto

Construir um sistema de autenticação/acesso utilizando:

- ESP32
- MFRC522 (RFID)
- MQTT
- API Python
- Dashboard Web
- LEDs de status

Fluxo principal:

ESP32 → MQTT → API Python → Banco → Resposta → LEDs

Estrutura Inicial do Projeto

```
RFID-ESP32-reader /
├── venv/
├── api/
│   ├── main.py
│   ├── config.py
│   ├── database.py
│   ├── mqtt_client.py
│   ├── models.py
│   └── routes/
│       └── access.py
├── dashboard/
├── esp32/
│   └── firmware.ino
├── .env
├── .gitignore
├── requirements.txt
└── README.md
```

1. Arquivo .gitignore

Crie:

```
.gitignore
```

Conteúdo:

```
venv/  
__pycache__/  
.env  
*.pyc
```

2. Arquivo .env

Crie:

```
.env
```

Conteúdo:

```
APP_NAME=RFID API  
APP_HOST=127.0.0.1  
APP_PORT=8000  
  
MQTT_HOST=localhost  
MQTT_PORT=1883  
MQTT_TOPIC=rfid/leitura  
  
DATABASE_URL=sqlite:///rfid.db
```

3. Instalar dependências

Com venv ativado:

```
pip install fastapi uvicorn sqlalchemy paho-mqtt python-dotenv
```

Depois:

```
pip freeze > requirements.txt
```

4. Configuração do Projeto

Arquivo:

```
api/config.py
```

```
import os
from dotenv import load_dotenv

load_dotenv()

APP_NAME = os.getenv("APP_NAME")
APP_HOST = os.getenv("APP_HOST")
APP_PORT = int(os.getenv("APP_PORT"))

MQTT_HOST = os.getenv("MQTT_HOST")
MQTT_PORT = int(os.getenv("MQTT_PORT"))
MQTT_TOPIC = os.getenv("MQTT_TOPIC")

DATABASE_URL = os.getenv("DATABASE_URL")
```

5. Banco de Dados

Arquivo:

```
api/database.py
```

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

from config import DATABASE_URL

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}
)

SessionLocal = sessionmaker(
```

```
        autocommit=False,  
        autoflush=False,  
        bind=engine  
    )  
  
    Base = declarative_base()
```

6. Modelos do Banco

Arquivo:

api/models.py

```
from sqlalchemy import Column, Integer, String, DateTime  
from datetime import datetime  
  
from database import Base  
  
class AccessLog(Base):  
    __tablename__ = "access_logs"  
  
    id = Column(Integer, primary_key=True, index=True)  
    uid = Column(String, nullable=False)  
    status = Column(String, nullable=False)  
    created_at = Column(DateTime, default=datetime.utcnow)
```

7. API Principal

Arquivo:

api/main.py

```
from fastapi import FastAPI  
  
from database import engine  
from models import Base  
  
Base.metadata.create_all(bind=engine)  
  
app = FastAPI()
```

```
        title="RFID API"
    )

@app.get("/")
def home():

    return {
        "status": "API ONLINE"
    }

@app.get("/health")
def health():

    return {
        "service": "ok"
    }
```

8. Executando a API

Entre na pasta:

```
cd api
```

Execute:

```
uvicorn main:app --reload
```

9. Testando

Acesse:

```
http://127.0.0.1:8000
```

Resultado esperado:

```
{
  "status": "API ONLINE"
}
```

10. Swagger Automático

FastAPI gera documentação automaticamente.

Acesse:

`http://127.0.0.1:8000/docs`

Dashboard em Tempo Real

Objetivo

Criar uma interface web em tempo real para apresentar:

- UID lido
- Nome do usuário
- Status do acesso
- Horário da leitura
- Histórico de acessos

Sem necessidade de recarregar a página.

Arquitetura Realtime

```
ESP32
  ↓ MQTT
FastAPI
  ↓
Banco SQLite
  ↓ WebSocket
Dashboard Web
```

Fluxo Completo

1. ESP32 lê RFID

Exemplo UID:

04A3F91C

2. ESP32 publica MQTT

Tópico:

rfid/leitura

Payload:

```
{  
  "uid": "04A3F91C"  
}
```

3. API recebe UID

A API:

- consulta banco
 - verifica usuário
 - registra log
 - envia resposta realtime
-

4. Dashboard recebe atualização

A tela atualiza automaticamente:

Autorizado

- ✓ Matheus
 - ✓ Acesso liberado
 - ✓ 21:40:12
-

Negado

- ✗ UID não cadastrado
 - ✗ Acesso negado
-

Estrutura Recomendada Dashboard

```
dashboard/  
|  
├─ index.html  
├─ style.css  
└─ app.js
```

Estrutura Recomendada Backend

```
api/  
|  
├─ websocket_manager.py  
├─ mqtt_client.py  
├─ services/  
│   └─ access_service.py  
└─ routes/  
    └─ rfid.py
```

Estrutura do Banco

Tabela users

Campo	Tipo
id	Integer
nome	String
uid	String
ativo	Boolean

Tabela access_logs

Campo	Tipo
id	Integer
uid	String
status	String

Campo	Tipo
created_at	DateTime

Tecnologias do Projeto

Camada	Tecnologia
Firmware	ESP32 Arduino
Comunicação	MQTT
Backend	FastAPI
Banco	SQLite
ORM	SQLAlchemy
Realtime	WebSocket
Frontend	HTML/CSS/JS

Próximas Etapas

Etapa 1

- Validar API
- Validar banco SQLite

Etapa 2

- Criar endpoint de leitura RFID

Etapa 3

- Integrar MQTT

Etapa 4

- Criar dashboard web

Etapa 5

- Integrar ESP32

Etapa 6

- Acionamento LEDs

Fluxo Futuro Completo

```
ESP32
↓
Leitura RFID
↓
Publica MQTT
↓
API recebe UID
↓
Banco consulta permissão
↓
API responde
↓
ESP32 acende LED verde/vermelho
```

Organização Recomendada

Sempre:

1. Desenvolver uma camada por vez
2. Testar isoladamente
3. Validar logs
4. Versionar no Git

Comandos Úteis

Ativar venv

```
.\venv\Scripts\Activate.ps1
```

Atualizar requirements

```
pip freeze > requirements.txt
```

Rodar API

```
uvicorn main:app --reload
```

Instalar requirements

```
pip install -r requirements.txt
```